EXPERIMENT - 6

Aim : To create and deploy a Ballot smart contract using Solidity and Remix IDE and perform voting and vote delegation transactions.

Theory :

1. Smart Contract

A **smart contract** is a program deployed on a blockchain that executes predefined rules automatically when its functions are called. In this experiment, the **Ballot smart contract** is used to create a blockchain-based voting system. It allows voters to receive voting rights, cast votes for proposals, and delegate their voting power to another voter.

2. require Statement

The require statement is used to verify whether a particular condition is true before executing a function. If the condition is false, the transaction is stopped and the changes made during that transaction are reverted.

In the Ballot contract, require is used to:

- Ensure that only the **chairperson** can assign voting rights.
- Check that a voter has the required voting rights.
- Prevent a voter from casting a vote more than once.
- Prevent a voter from delegating their vote to themselves.
- Check whether voting rights have already been assigned.

Therefore, require helps to maintain the **security, correctness, and proper functioning** of the voting system.

3. Mapping

A **mapping** is a Solidity data structure that stores data in the form of **key-value pairs**.

Syntax:

mapping(address => Voter) public voters;

In the Ballot contract, the Ethereum **address of a voter** is used as the key, and the corresponding **Voter structure** is stored as the value.

It keeps information such as:

- Voting weight

- Voting status
- Delegate address
- Proposal selected by the voter

Mappings allow voter information to be accessed efficiently using their Ethereum address.

4. Storage

Storage is the permanent data storage area of a smart contract on the blockchain. Variables stored in storage remain available even after the function execution is completed and can be accessed during future transactions.

For example, important voter and proposal information in the Ballot contract is stored permanently on the blockchain.

5. Memory

Memory is temporary storage used during the execution of a function. Data stored in memory exists only while the function is executing.

For example:

constructor(string[] memory proposalNames)

Here, proposalNames is temporarily stored in memory while the constructor is running. After the function execution is completed, memory data is discarded.

6. bytes32 and string

The Ballot contract can use the bytes32 data type for storing proposal names.

bytes32 name;

bytes32 is a fixed-size data type that can store up to **32 bytes** of data. It is suitable for storing short text values and can be efficient for blockchain operations.

However, because it has a fixed size, it is not suitable for storing longer text. A string can be used when more flexible text storage is required.

7. Ballot Contract

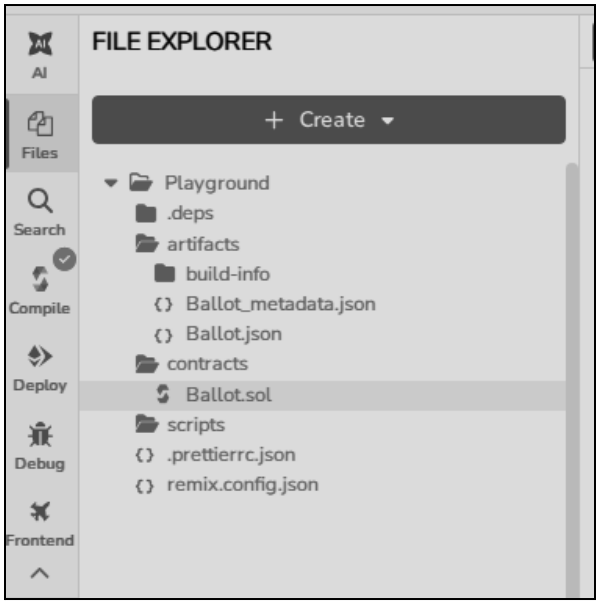
The **Ballot contract** implements the main operations required for a blockchain-based voting system. It contains:

- **Voter structure** – stores information related to each voter.

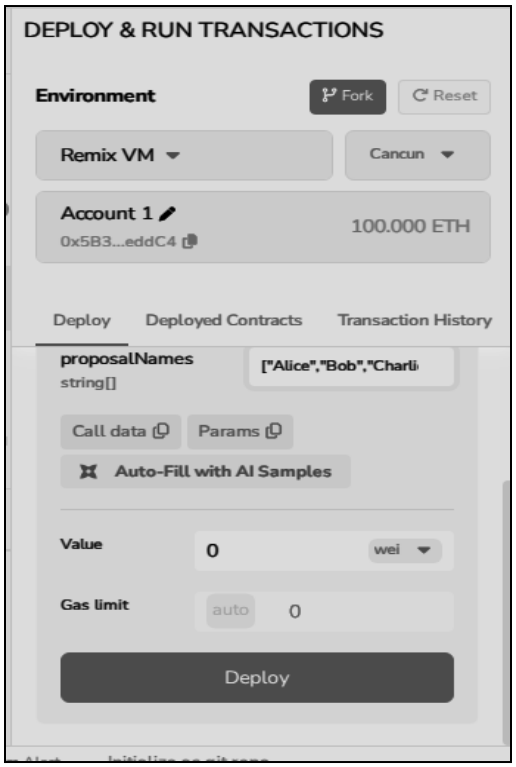
- **Proposal structure** – stores proposal names and their vote counts.
- **Chairperson** – identifies the account responsible for assigning voting rights.
- **Voters mapping** – maintains voter information using Ethereum addresses.
- **Proposals array** – stores the available proposals.
- **giveRightToVote()** – assigns voting rights to a voter.
- **delegate()** – allows a voter to transfer/delegate voting power to another voter.
- **vote()** – allows a voter to vote for a proposal.
- **winningProposal()** – determines the proposal with the highest votes.
- **winnerName()** – returns the name of the winning proposal.

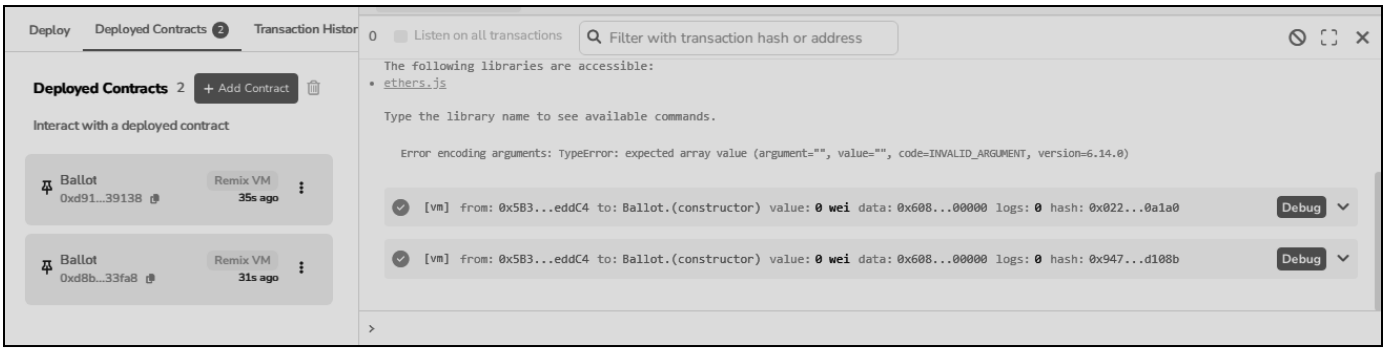
Thus, the Ballot smart contract demonstrates how **Solidity, blockchain storage, mappings, structures, and smart contract functions** can be combined to implement a decentralized voting mechanism.

Implementation :

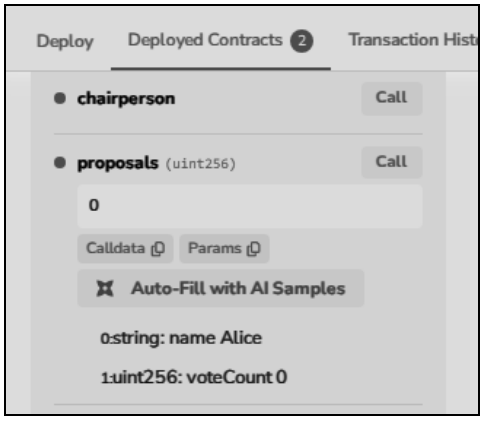


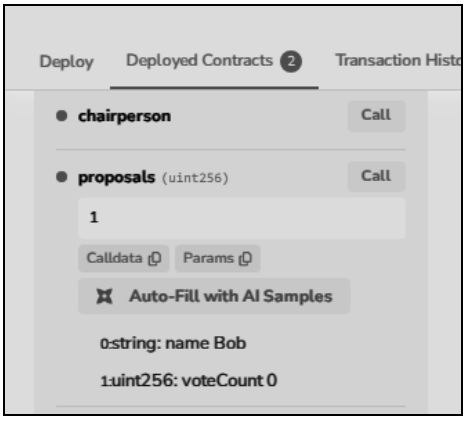
The Ballot smart contract was successfully compiled in Remix IDE without compilation errors.

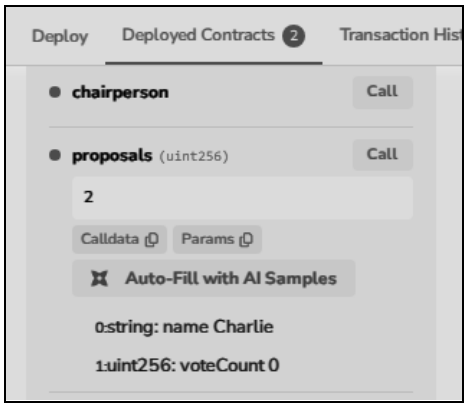




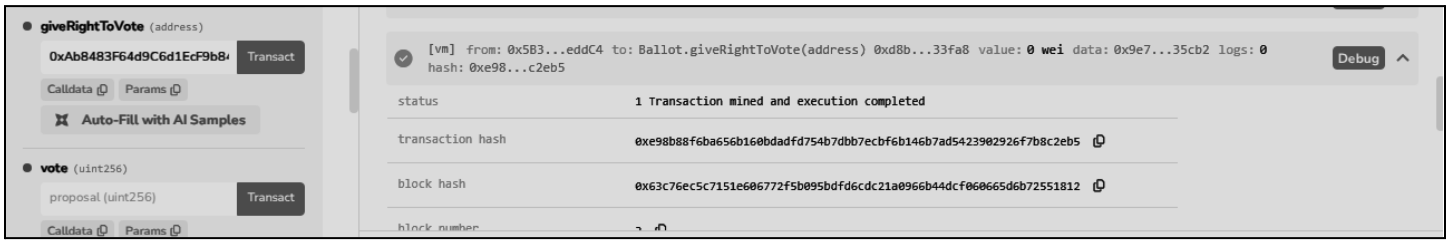
Three proposal candidates, Alice, Bob and Charlie, were successfully loaded into the Ballot contract.



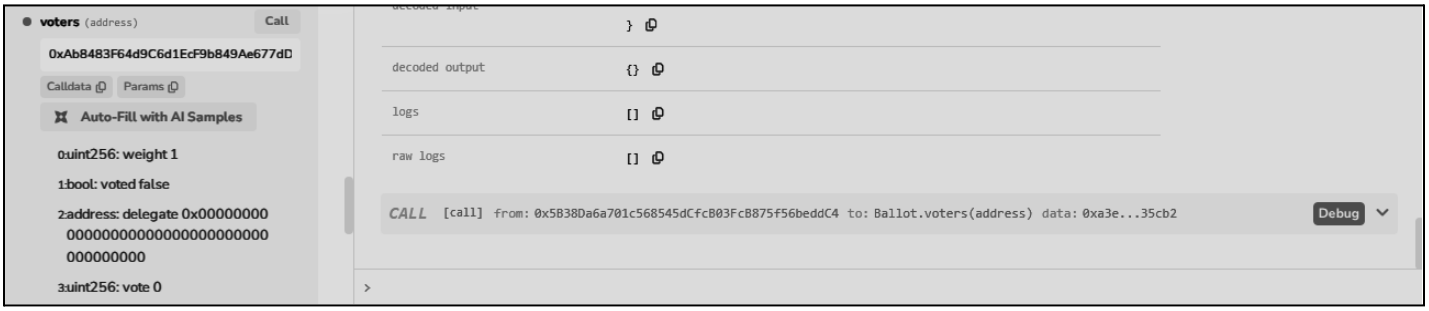


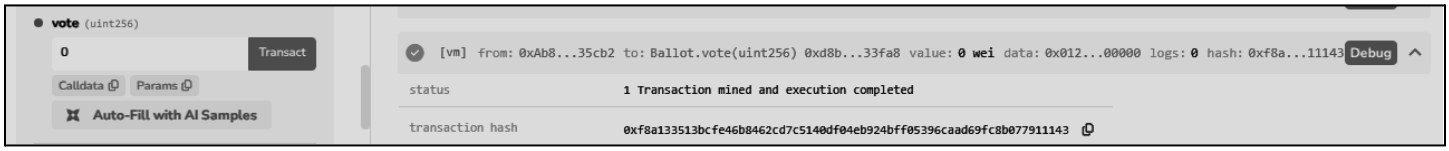


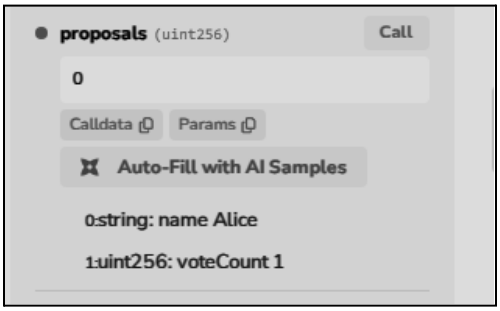
Proposal 0 corresponds to Alice,proposal 1 corresponds to Bob and proposal 2 corresponds to Charlie.

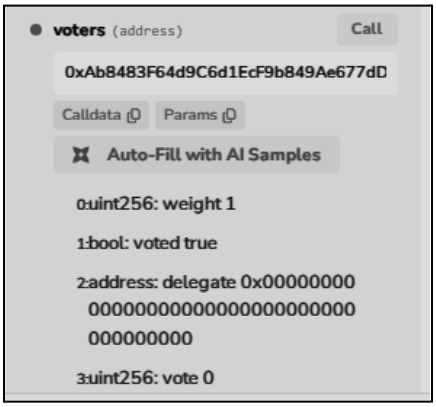


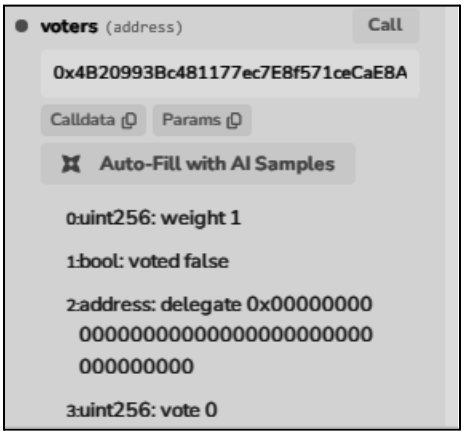
The chairperson successfully granted voting rights to another account.

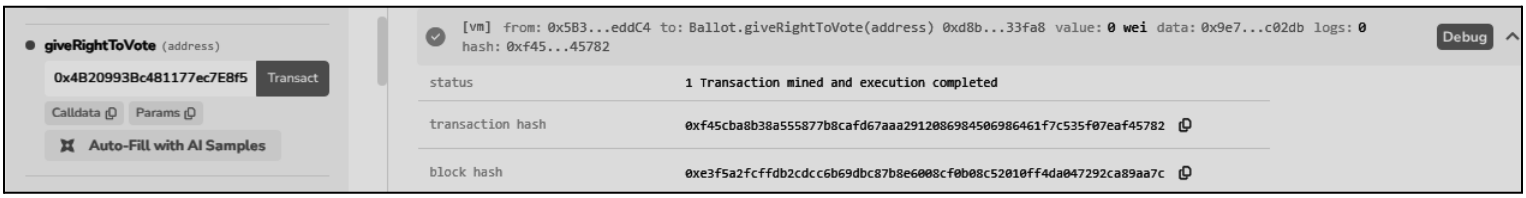




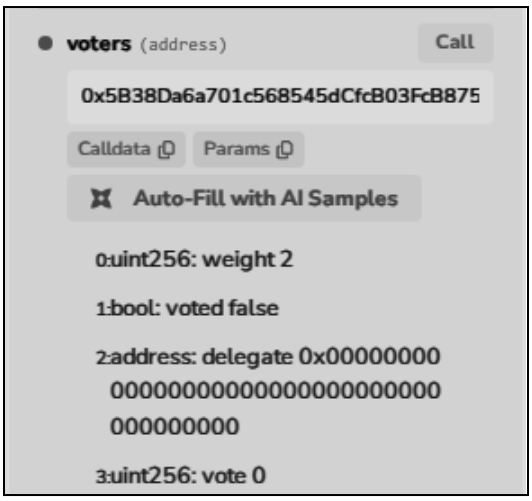


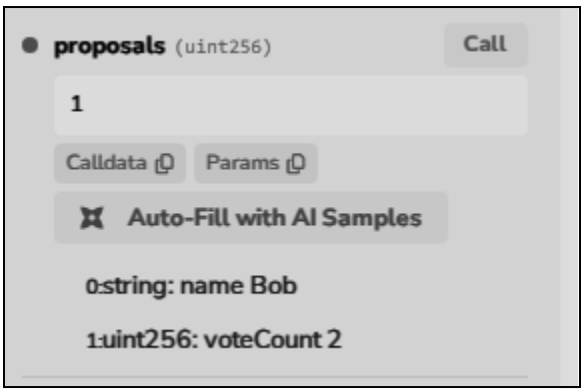


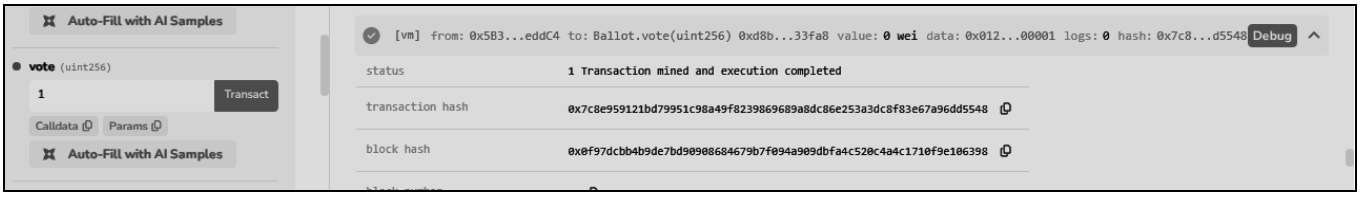


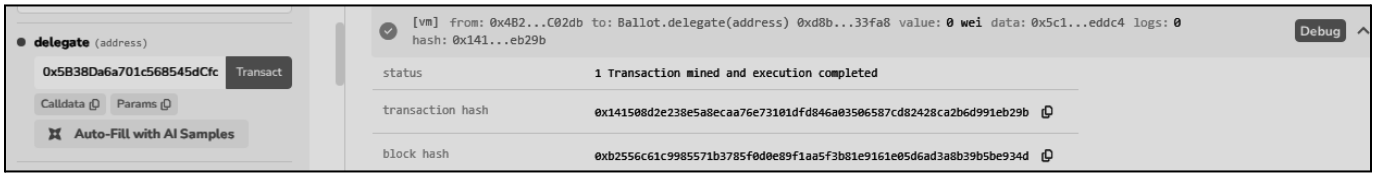


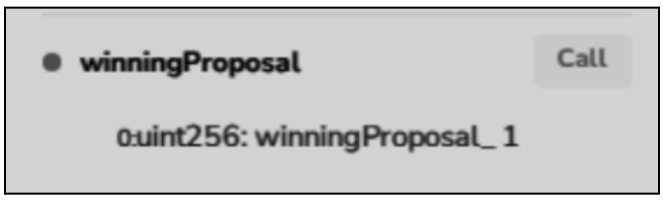
After voting, the voter's status changed to voted = true, confirming that the vote was recorded successfully. Account 3 successfully delegated its voting power to Account 1.











Conclusion : Thus, the creation and deployment of a Ballot smart contract using Solidity and Remix IDE was successfully performed. The experiment helped in understanding smart contract deployment, transactions, require statements, mappings, storage and memory, voting, vote delegation, and the difference between bytes32 and string. The successful execution demonstrated how blockchain-based voting can record and manage voting information securely and transparently.